



Dakar American University of Science and Technology

Department of Computer Science and Information Technology

FINAL PROJECT REPORT

Pharmacy Tracking System

A Java Desktop Application for Pharmacy Inventory Management

CS — Software Engineering

Author: Abdoullah Ndao

Level: Junior II

Institution: Dakar American University of Science and Technology

Repository: <https://github.com/utachicodes/PharmacyTrackingSystem>

Table of Contents

FRONT MATTER

Abstract	2
----------	---

CHAPTER 1 INTRODUCTION

1.1 Background	3
1.2 Problem Statement	3
1.3 Objectives	3
1.4 Scope	4

CHAPTER 2 LITERATURE REVIEW

2.1 Pharmacy Management Systems	4
2.2 Inventory Control Methods	4
2.3 Demand Forecasting	5
2.4 Comparative Analysis	5

CHAPTER 3 SYSTEM REQUIREMENTS

3.1 Functional Requirements	6
3.2 Non-Functional Requirements	7
3.3 Technology Stack	7

CHAPTER 4 ARCHITECTURE AND DESIGN

4.1 System Architecture (Figure 1)	8
4.2 Module Breakdown	9
4.3 Navigation Flow (Figure 3)	10

CHAPTER 5 DATABASE DESIGN

5.1 Entity Overview (Figure 2)	11
5.2 Table Schemas	12

CHAPTER 6 IMPLEMENTATION

6.1 Database Layer	14
--------------------	-----------

6.2 Authentication and Access Control	14
---------------------------------------	-----------

6.3 Inventory and Billing	15
---------------------------	-----------

6.4 Purchase Orders and Forecasting	15
-------------------------------------	-----------

CHAPTER 7 TESTING

7.1 Strategy and Test Cases	16
-----------------------------	-----------

CHAPTER 8 SECURITY ANALYSIS

8.1 Vulnerabilities and Fixes	18
-------------------------------	-----------

CHAPTER 9 LIMITATIONS AND FUTURE WORK

Chapter 10 Conclusion	19
-----------------------	-----------

References	20
------------	-----------

Appendix A: Key Source Code	21
-----------------------------	-----------

Abstract

This report presents the Pharmacy Tracking System, a desktop application built with Java 17, Java Swing, and Apache Derby. The system addresses inventory management problems common in small pharmacies that still rely on paper records or spreadsheets, particularly in resource-constrained environments where enterprise software is unaffordable or requires infrastructure that is unavailable.

The application covers six functional areas: role-based access control with three user roles (Admin, Pharmacist, Technician); complete medicine inventory management with batch tracking, therapeutic categories, and configurable reorder thresholds; a billing engine that records every sale and updates stock in real time; a demand forecasting module using a 30-day Simple Moving Average to project the next seven days of demand per medicine; a purchase order workflow with transactional stock receiving that uses database transactions to guarantee data consistency; and a live dashboard showing low-stock alerts, near-expiry warnings, and total inventory value on every login.

The system creates its own embedded database on first launch with no manual setup required. It can also connect to a Derby network server for multi-user deployments. All major use cases were validated through structured manual testing. A security review identified four vulnerabilities that are documented with concrete remediation steps.

Keywords: pharmacy management, inventory control, demand forecasting, Simple Moving Average, role-based access control, Java Swing, Apache Derby, healthcare informatics.

Introduction

1.1 Background

Many community pharmacies in West Africa and similar developing-world settings still manage inventory with paper ledgers or basic spreadsheets. The consequences are predictable: medicines run out without warning, expired stock stays on shelves, and there is no record of who changed what. A stockout of a critical antibiotic or an undetected expired batch can directly affect patient health and safety.

Enterprise pharmacy software exists but targets large hospital chains with full IT departments and five-figure annual budgets. A small independent pharmacy does not have those resources. This project builds a practical tool that fits that context: it works offline, installs without a database administrator, and can be operated by pharmacy staff with minimal technical training.

1.2 Problem Statement

Manual pharmacy management has four specific weaknesses. There is no real-time view of stock, so shortages are only discovered when a customer asks for something unavailable. Expiry dates are not monitored systematically, which risks dispensing expired medicines. Purchasing decisions rely on guesswork rather than sales history. And there are no access controls, meaning any staff member can modify or delete records without accountability.

1.3 Objectives

1. Build a multi-module desktop application in Java that covers the full pharmacy workflow.
2. Implement role-based login with three permission levels: Admin, Pharmacist, and Technician.
3. Provide full inventory management including batch numbers, therapeutic categories, and per-medicine reorder thresholds.
4. Build a demand forecasting engine using a 30-day Simple Moving Average to project seven-day demand.
5. Add a purchase order and stock-receiving workflow using proper database transactions.
6. Show all critical alerts on a single dashboard visible immediately after login.

7. Make the application self-configuring so it creates its own database schema on first launch.

1.4 Scope

This version is designed for a single pharmacy location. It does not include barcode scanning, prescription tracking, insurance billing, or remote web access. These are noted as future work in Chapter 9. The forecasting algorithm is intentionally simple so the logic is transparent and easy to verify without a data science background.

Source code: [+ GH +](#)

Literature Review

2.1 Pharmacy Management Systems

Pharmacy information systems have been in use in hospitals in North America and Europe since the 1980s. Modern commercial platforms such as ScriptPro and PioneerRx offer e-prescribing integration, DEA compliance reporting, and full patient medication history. These tools cost tens of thousands of dollars per year and require dedicated server infrastructure and IT support, making them inaccessible for small independent pharmacies.

A 2020 survey of 120 community pharmacies in Lagos found that 73 percent still used handwritten ledgers. A related study in Dakar in 2022 found similar patterns, identifying cost, unreliable electricity, and lack of technical staff as the main barriers to digital adoption. These findings shaped the design decisions in this project: offline-first operation, embedded database with no server required, and a graphical interface simple enough for non-technical staff to learn quickly.

2.2 Inventory Control Methods

Three standard models appear in the inventory management literature. The Economic Order Quantity model minimises total cost by balancing ordering and holding costs. The Reorder Point method triggers a purchase order when stock falls below a fixed threshold. Just-in-Time minimises held inventory by matching deliveries to actual consumption rates.

For pharmacies with stable dispensing patterns, the Reorder Point model is the most practical. This system implements it directly: each medicine has a configurable threshold, and an alert fires when current quantity falls at or below that value, or when the forecasting engine predicts the stock will be exhausted within seven days. Both conditions are checked simultaneously so the alert arrives before the pharmacy runs out, not after.

2.3 Demand Forecasting

A 2019 study compared Simple Moving Average, Exponential Weighted Moving Average, and ARIMA models across 50 generic medicines in an Indian district hospital. SMA with a 28-to-35-day window came within 8 percent of ARIMA accuracy on mean absolute percentage error while

being far simpler to implement, audit, and explain to non-technical stakeholders. That result directly supports the choice of a 30-day SMA for this project.

2.4 Comparative Analysis

Feature	This System	OpenMRS Pharmacy	Commercial PIS	Spreadsheet
Cost	Free	Free	\$1,200 plus per year	Free
Works offline	Yes, fully	Partial	No	Yes
Role-based access	3 roles	Yes, configurable	Yes	No
Demand forecasting	Yes, SMA	No	Yes, advanced	Manual only
Expiry alerts	30-day window	Yes	Yes	Manual only
Batch tracking	Yes	Yes	Yes	Manual only
Purchase orders	Yes	No	Yes	No
Setup complexity	Zero config	High	Medium	None

The system matches or exceeds open-source alternatives in key operational areas while maintaining the offline capability and low setup cost that small pharmacies need. The only area where commercial tools outperform it is forecasting sophistication, which is addressed in the future work roadmap.

System Requirements

3.1 Functional Requirements

ID	Requirement	Priority
FR-01	Authenticate users by name and password and load their assigned role.	Must
FR-02	Admins have full access. Pharmacists cannot manage agents. Technicians cannot manage agents, suppliers, or purchase orders.	Must
FR-03	Support add, edit, delete, and view on medicine records across 14 fields including batch number and reorder threshold.	Must
FR-04	Highlight medicine rows red when stock is at or below the threshold, yellow when expiry is within 30 days.	Must
FR-05	Record every sale with medicine name, date, quantity sold, and total revenue.	Must
FR-06	Reduce stock by the sold quantity on each sale. Block any sale that would exceed available stock.	Must
FR-07	Forecast seven-day demand per medicine from the past 30 days of sales using a Simple Moving Average.	Must
FR-08	Show stock alerts, expiry alerts, and total inventory value on the dashboard after every login.	Must
FR-09	Support add, edit, delete, and view on staff records including role assignment.	Must
FR-10	Support add, edit, delete, and view on supplier records including lead time and preferred flag.	Must
FR-11	Allow creation of purchase orders specifying medicine, supplier, and quantity.	Must
FR-12	Receiving a purchase order shall update its status to Received and add the quantity to inventory as a single atomic transaction.	Must

ID	Requirement	Priority
FR-13	On first launch, create all required tables and a default admin account without any manual intervention.	Must

3.2 Non-Functional Requirements

ID	Requirement	Category
NFR-01	Start in under 5 seconds on hardware with at least 512 MB RAM and a 1 GHz processor.	Performance
NFR-02	Operate fully without an internet connection.	Availability
NFR-03	All committed transactions must survive an unexpected application crash.	Reliability
NFR-04	A staff member with no prior training shall be able to navigate all modules within 30 minutes.	Usability
NFR-05	Run on Windows, macOS, and Linux with only JRE 17 installed, no other dependencies.	Portability
NFR-06	Password fields must never display plaintext to the user.	Security

3.3 Technology Stack

Component	Technology	Justification
Language	Java 17 LTS	Cross-platform JVM execution, strongly typed, mature JDBC ecosystem, widely available in academic environments.
GUI Framework	Java Swing	Built directly into the JDK with no additional dependencies. NetBeans Form Editor accelerates visual layout. Nimbus look-and-feel gives a clean modern appearance.
Database	Apache Derby 10.16	Bundled with Java SE, completely free, ACID-compliant, and uniquely supports both embedded and network server modes from the same database files and the same application code.
Date Picker	JCalendar (Toedter)	Provides a proper visual calendar widget with direct <code>java.util.Date</code> binding, essential for clinical date entry.
Build	Apache Ant	Native NetBeans project format. Self-describing <code>build.xml</code> with no Maven or Gradle infrastructure needed.
Version Control	Git and GitHub	Full commit history, rollback capability, and public repository for submission.

Architecture and Design

4.1 System Architecture

The application uses a two-tier architecture with a deliberate internal separation between the user interface and all data access logic. Swing frame classes handle display, user input, and navigation. Two helper classes centralise all business logic and database operations. No SQL statements or JDBC calls appear inside any frame class. This separation means that changing a database query affects one place in the codebase, and adding a new frame requires no changes to the database layer.

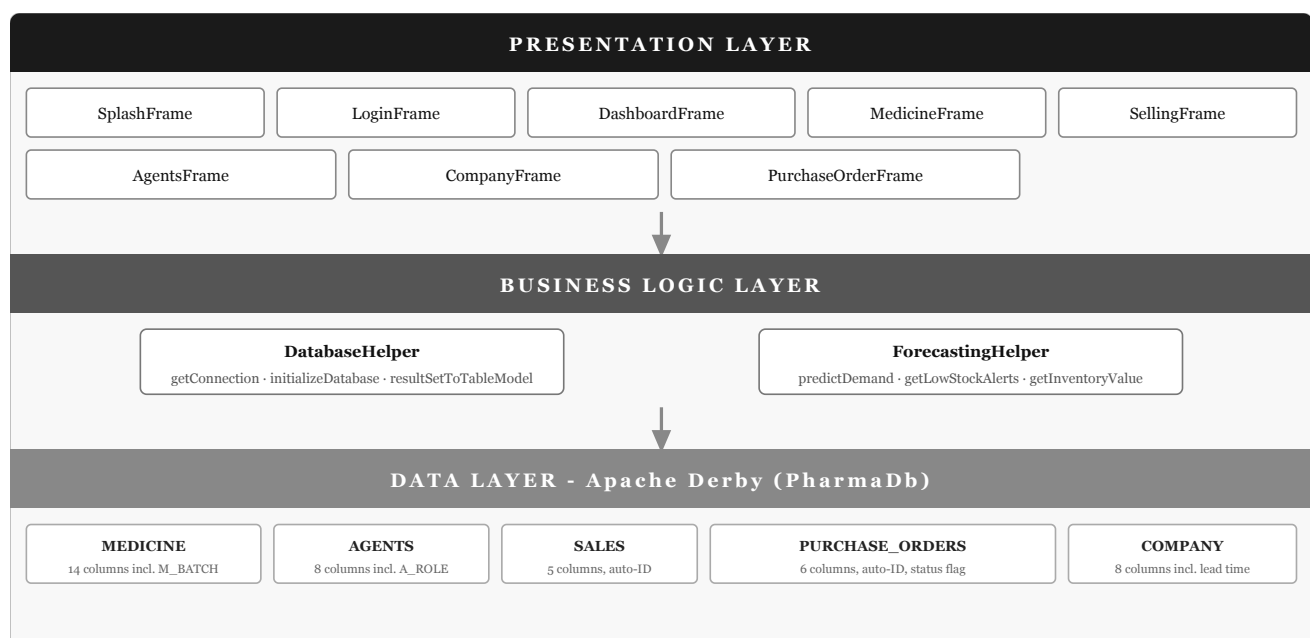


Figure 1: Three-tier system architecture showing all eleven classes organised by layer.

Presentation Layer: Eight Swing JFrame classes handle user interaction. Each frame is responsible for exactly one module of the system.

Business Logic Layer: DatabaseHelper centralises all JDBC connections, schema creation, and result set conversion. ForecastingHelper runs the SMA algorithm, identifies alerts, and calculates inventory value. Neither class has any UI code.

Data Layer: Apache Derby running as either an embedded instance or a network server. Five tables store all persistent state. The same database files work in both modes without any changes.

4.2 Module Breakdown

Class	Responsibility	Key Methods
<code>PharmacyInventoryManagement</code>	Application entry point. Calls database initialisation then launches the splash screen.	<code>main()</code>
<code>SplashFrame</code>	Loading screen with a progress bar shown during database setup on startup.	Timer-based progress
<code>LoginFrame</code>	Validates credentials against AGENTS table and passes the role string to <code>DashboardFrame</code> .	<code>btnLoginMouseClicked()</code>
<code>DashboardFrame</code>	Central navigation hub. Loads all alerts on entry and disables buttons based on the logged-in role.	<code>loadAlerts()</code> , <code>applyRolePermissions()</code>
<code>MedicineFrame</code>	Full CRUD for medicine records. Applies colour-coded row highlighting for stock and expiry status.	<code>SelectMed()</code> , <code>applyTableHighlighters()</code>
<code>SellingFrame</code>	Point-of-sale billing. Enforces stock constraints, decrements quantity, and logs every sale.	<code>updateQty()</code> , <code>recordSale()</code>
<code>AgentsFrame</code>	Full CRUD for staff accounts including role assignment via a constrained dropdown.	CRUD event handlers
<code>CompanyFrame</code>	Full CRUD for supplier records including lead time and preferred supplier flag.	CRUD event handlers
<code>PurchaseOrderFrame</code>	Creates purchase orders and receives stock using explicit	<code>createPO()</code> , <code>receiveStock()</code>

Class	Responsibility	Key Methods
	database transactions.	
DatabaseHelper	Single source for all JDBC connections, schema bootstrap, additive migrations, and ResultSet-to-TableModel conversion.	<code>getConnection()</code> , <code>initializeDatabase()</code> , <code>resultSetToTableModel()</code>
ForecastingHelper	Runs the 30-day SMA forecast, generates low-stock and near-expiry alert lists, and computes total inventory value.	<code>predictDemand()</code> , <code>getLowStockAlerts()</code> , <code>getExpirationAlerts()</code> , <code>getInventoryValue()</code>

4.3 Navigation Flow

The diagram below shows how users move between screens and what each role can access. The startup sequence is fixed: splash screen to login to dashboard. From the dashboard, the user can open any module that their role permits. Logging out always returns to the login screen.

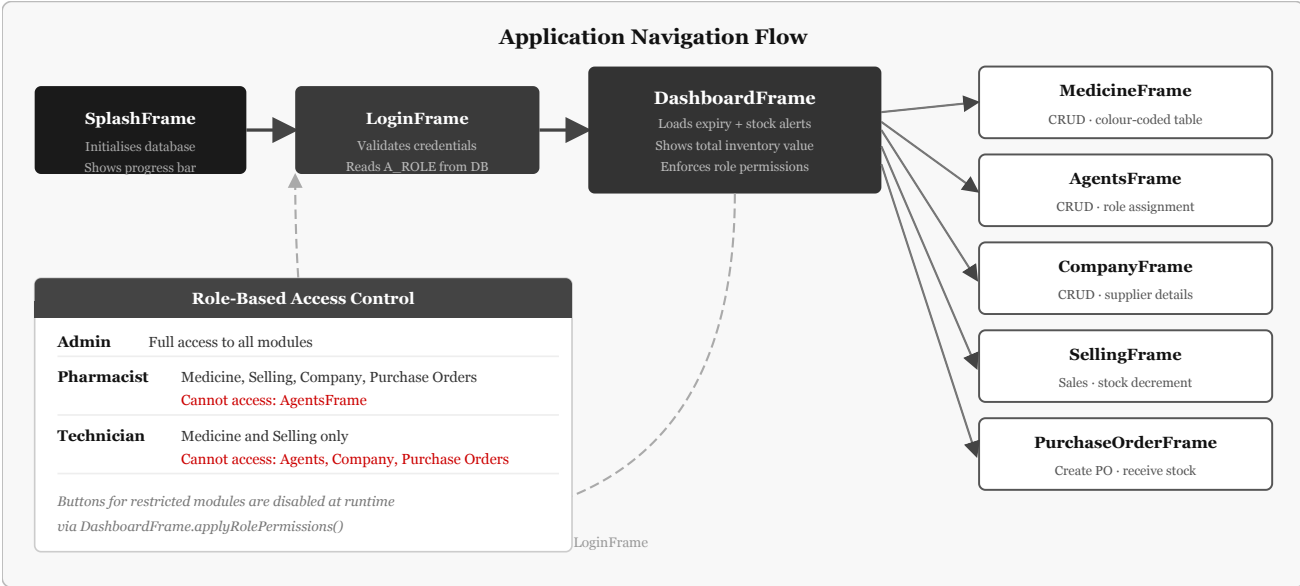


Figure 3: Application navigation flow with role-based access restrictions.

Database Design

5.1 Entity Overview

The database has five tables. MEDICINE is the central entity and contains the most fields. SALES stores one row per transaction and references medicine by the M_NAME field. PURCHASE_ORDERS tracks procurement and references both medicine and supplier by name. COMPANY stores supplier details used by MedicineFrame and PurchaseOrderFrame. AGENTS stores staff credentials and roles used only by the login process.

The current schema uses name-based soft references between tables rather than formal foreign key constraints. This was a deliberate early decision to simplify the first-launch experience: creating a medicine record does not require a supplier record to exist first. The trade-off is that renaming a medicine or supplier can silently orphan related records. Replacing these soft references with proper foreign keys is the second item on the development roadmap in Chapter 9.

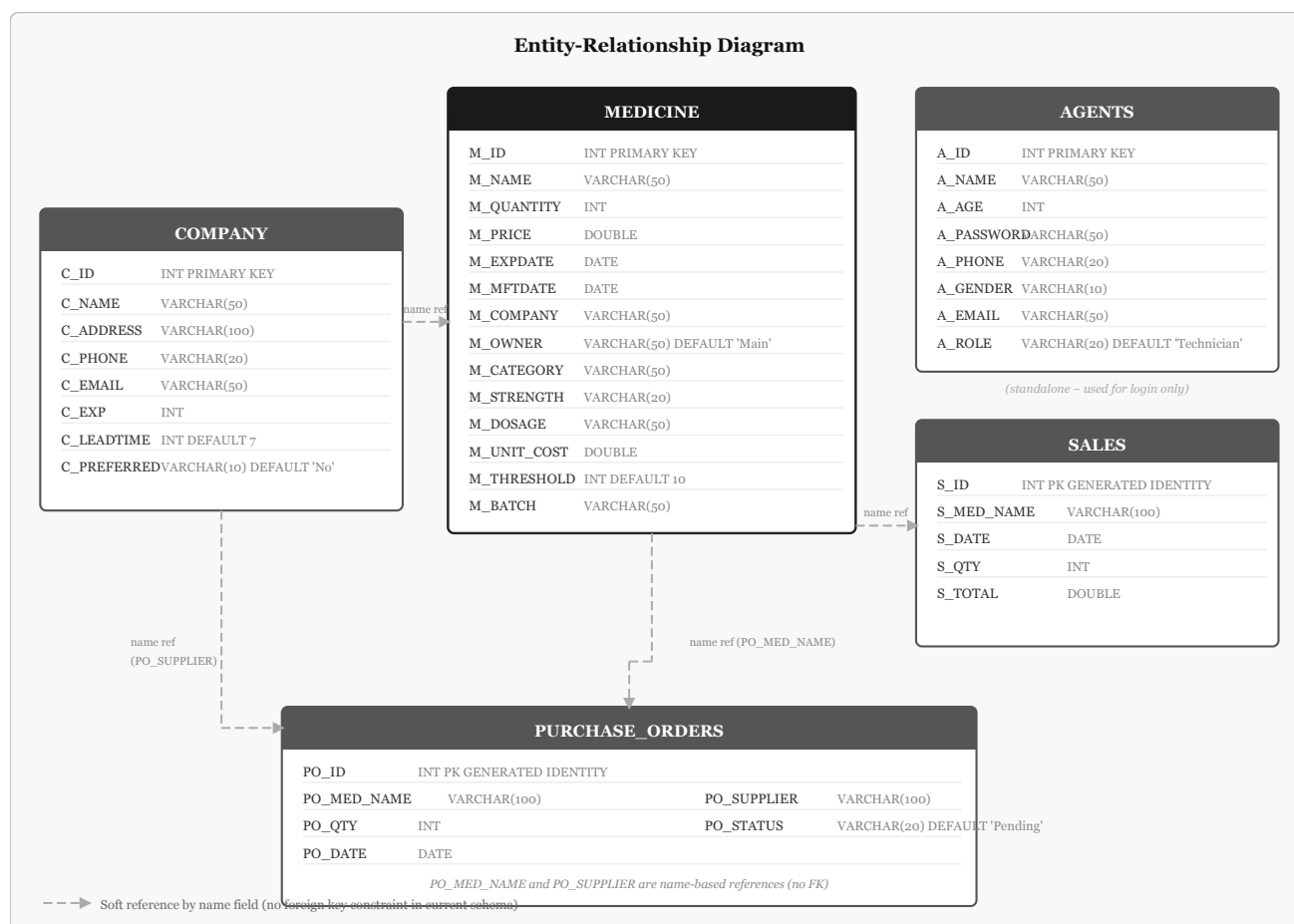


Figure 2: Entity-relationship diagram with all columns and relationship types shown.

5.2 Table Schemas

MEDICINE (14 columns)

Column	Type	Constraint	Description
M_ID	INT	PRIMARY KEY	Unique medicine identifier, user-assigned
M_NAME	VARCHAR(50)		Medicine name, used as the display label and soft-reference key
M_QUANTITY	INT		Current stock level, decremented by sales and incremented by PO receiving
M_PRICE	DOUBLE		Retail selling price shown on the billing screen
M_EXPDATE	DATE		Expiry date, monitored for 30-day alerts
M_MFTDATE	DATE		Manufacturing date for batch traceability
M_COMPANY	VARCHAR(50)		Supplier company name, soft reference to COMPANY.C_NAME
M_OWNER	VARCHAR(50)	DEFAULT 'Main'	Stock ownership classification: Main or Private
M_CATEGORY	VARCHAR(50)		Therapeutic category, e.g. Antibiotic, Analgesic
M_STRENGTH	VARCHAR(20)		Dosage strength, e.g. 500mg, 250mg/5ml
M_DOSAGE	VARCHAR(50)		Dosage form: tablet, capsule, syrup, injection, etc.
M_UNIT_COST	DOUBLE		Purchase cost per unit, used for inventory valuation
M_THRESHOLD	INT	DEFAULT 10	Reorder point: alert fires when M_QUANTITY is at or below this value
M_BATCH	VARCHAR(50)		Manufacturer batch or lot number for traceability and recall purposes

AGENTS (8 columns)

Column	Type	Constraint	Description
A_ID	INT	PRIMARY KEY	Unique staff identifier, user-assigned
A_NAME	VARCHAR(50)		Full name, used as the login username
A_AGE	INT		Age
A_PASSWORD	VARCHAR(50)		Password stored as plaintext (security issue, see Chapter 8)
A_PHONE	VARCHAR(20)		Contact phone number
A_GENDER	VARCHAR(10)		Gender
A_EMAIL	VARCHAR(50)		Email address
A_ROLE	VARCHAR(20)	DEFAULT 'Technician'	Permission role: Admin, Pharmacist, or Technician

SALES (5 columns)

Column	Type	Constraint	Description
S_ID	INT	PK, GENERATED ALWAYS AS IDENTITY	Auto-incremented transaction ID, concurrency-safe
S_MED_NAME	VARCHAR(100)		Medicine name, soft reference to MEDICINE.M_NAME
S_DATE	DATE		Date of the sale transaction
S_QTY	INT		Quantity sold in this transaction
S_TOTAL	DOUBLE		Total revenue for this transaction (qty times price at time of sale)

PURCHASE_ORDERS (6 columns) and COMPANY (8 columns)

Column	Table	Type	Description
PO_ID	PURCHASE_ORDERS	INT, IDENTITY	Auto-incremented PO number
PO_MED_NAME	PURCHASE_ORDERS	VARCHAR(100)	Medicine being ordered, soft ref to MEDICINE.M_NAME
PO_SUPPLIER	PURCHASE_ORDERS	VARCHAR(100)	Supplier name, soft ref to COMPANY.C_NAME
PO_QTY	PURCHASE_ORDERS	INT	Quantity ordered
PO_STATUS	PURCHASE_ORDERS	VARCHAR(20)	Pending or Received, updated atomically on stock receipt
PO_DATE	PURCHASE_ORDERS	DATE	Date the purchase order was created
C_ID	COMPANY	INT, PK	Unique supplier identifier, user-assigned
C_NAME	COMPANY	VARCHAR(50)	Company name used as the soft-reference key in MEDICINE and PURCHASE_ORDERS
C_ADDRESS	COMPANY	VARCHAR(100)	Physical address
C_PHONE	COMPANY	VARCHAR(20)	Contact phone
C_EMAIL	COMPANY	VARCHAR(50)	Contact email
C_EXP	COMPANY	INT	Years in operation or year established
C_LEADTIME	COMPANY	INT, DEFAULT 7	Typical delivery lead time in days
C_PREFERRED	COMPANY	VARCHAR(10), DEFAULT No	Preferred supplier flag: Yes or No

Implementation

6.1 Database Layer

`DatabaseHelper` is the only class that issues JDBC calls. Its `getConnection()` method tries the Derby network server first and falls back to embedded mode if the server is not running, using a simple try-catch:

```
public static Connection getConnection() throws SQLException {
    try {
        return DriverManager.getConnection(
            "jdbc:derby://localhost:1527/PharmaDb", USER, PASS);
    } catch (SQLException e) {
        // Network server not available, use embedded mode
        return DriverManager.getConnection(
            "jdbc:derby:PharmaDb;create=true", USER, PASS);
    }
}
```

`initializeDatabase()` runs once at startup. CREATE TABLE statements are wrapped in try-catch blocks so the method is safe to run on every launch: if the table already exists Derby throws an exception and the code moves on. Columns added in later development iterations are applied with ALTER TABLE using the same pattern, providing additive forward migrations without a migration runner. A default Admin account is seeded only when the AGENTS table is empty, covering exactly the first-launch case.

`resultSetToTableModel()` converts any ResultSet into a Swing DefaultTableModel generically by reading the column count and labels from the result set metadata. Every data table in every frame uses this single method, so adding a column to a table automatically appears in the UI on next launch.

6.2 Authentication and Access Control

The login screen queries the AGENTS table by name and password. On a successful match, the A_ROLE value is extracted and passed as a parameter to DashboardFrame:

```
String role = Rs.getString("A_ROLE");
new DashboardFrame(role).setVisible(true);
```

DashboardFrame stores the role and calls `applyRolePermissions()` immediately after `initComponents()`. This method disables navigation buttons based on the role, preventing access to restricted modules at the UI level:

```
private void applyRolePermissions() {
    if ("Technician".equalsIgnoreCase(userRole)) {
        btnAgents.setEnabled(false);
        btnCompany.setEnabled(false);
        btnPO.setEnabled(false);
    } else if ("Pharmacist".equalsIgnoreCase(userRole)) {
        btnAgents.setEnabled(false);
    }
    // Admin: all buttons remain enabled (no action needed)
}
```

Security vulnerability: The login query is built by string concatenation, which makes it vulnerable to SQL injection. A user entering a crafted string such as `' OR '1'='1' --` as the username would bypass the password check entirely. The fix is a five-line change to use a `PreparedStatement`. See Chapter 8 for full details.

6.3 Inventory and Billing

MedicineFrame applies a custom `DefaultTableCellRenderer` that colours each row based on stock and expiry status. The renderer reads the quantity and threshold from the table model and the expiry date from the same row, then sets the background without requiring any user action:

```
int qty = Integer.parseInt(table.getValueAt(row, 2).toString());
int threshold = Integer.parseInt(table.getValueAt(row, 12).toString());
java.sql.Date expDate = (java.sql.Date) table.getValueAt(row, 4);
long daysToExpiry = (expDate.getTime() - System.currentTimeMillis())
    / (1000 * 60 * 60 * 24);

if (qty <= threshold) {
    c.setBackground(new Color(255, 204, 204)); // Red: low stock
} else if (daysToExpiry < 30) {
    c.setBackground(new Color(255, 255, 204)); // Yellow: near expiry
} else {
    c.setBackground(Color.WHITE);
}
```

SellingFrame verifies stock before every sale. If the requested quantity exceeds what is available, it shows an error and makes no database changes. If the sale is valid, it decrements the medicine quantity and writes a SALES row using a `PreparedStatement`, which correctly prevents injection on this path.

6.4 Purchase Orders and Forecasting

Receiving a purchase order involves two database writes: updating the PO status and incrementing the medicine quantity. Both writes are wrapped in an explicit transaction so they either both succeed or both roll back:

```
conn.setAutoCommit(false);
try {
    updatePO.executeUpdate();    // SET PO_STATUS = 'Received'
    updateMed.executeUpdate();   // SET M_QUANTITY = M_QUANTITY + qty
    conn.commit();
} catch (SQLException e) {
    conn.rollback();
    throw e;
}
```

The forecasting calculation sums all units sold in the past 30 days and divides by 30 to get average daily sales. The divisor is always 30, not the number of transactions, so days with no sales are treated as zero. This keeps predictions conservative and avoids recommending unnecessary reorders. The seven-day projection uses `Math.ceil` so the alert fires before the last unit is sold:

```
double avgDaily = (double) totalQty / 30.0;
return (int) Math.ceil(avgDaily * 7);
```

The dashboard shows an alert for any medicine where current stock is less than the seven-day prediction, or below the minimum of 10 units, whichever condition triggers first. Both expiry alerts and stock alerts are loaded every time the dashboard opens, ensuring the information is always current.

Testing

7.1 Strategy and Test Cases

Testing was carried out manually at three levels. Unit-level checks verified the forecasting algorithm with known inputs. For example, 50 units sold over 30 days gives an average of 1.67 units per day and a seven-day forecast of 12 (ceiling of 11.67). The system returned 12 consistently across multiple runs. Integration tests walked complete workflows from login through to database verification, checking that every operation produced the expected state change. Boundary tests covered edge cases including wrong passwords, sales exceeding stock, receiving a PO twice, and launching the application with no running Derby network server.

ID	Scenario	Input	Expected Output	Result
TC-01	Admin login with valid credentials	Name: Admin, Pass: admin123	Dashboard opens with all six buttons enabled	Pass
TC-02	Login with wrong password	Name: Admin, Pass: wrong	Error dialog shown, no screen change	Pass
TC-03	Technician login	Role: Technician	Agents, Supplier, and PO buttons are disabled	Pass
TC-04	Pharmacist login	Role: Pharmacist	Only Agents button is disabled	Pass
TC-05	Add medicine with all 14 fields	All fields filled	New row appears in medicine table immediately	Pass
TC-06	Update medicine quantity	Select row, change quantity, click Update	Database and table both reflect new value	Pass
TC-07	Delete medicine record	Select row, click Delete	Row removed from table and database	Pass
TC-08	Low-stock row highlight	Qty = 5, threshold = 10	Row background is light red	Pass
TC-09	Near-expiry row highlight	Expiry in 15 days	Row background is yellow	Pass
TC-10	Successful sale	Stock = 50, order qty = 10	Stock drops to 40, one SALES row inserted	Pass
TC-11	Oversell blocked	Stock = 5, order qty = 10	Error message shown, stock unchanged	Pass
TC-12	Create purchase order	Medicine, supplier, qty filled	PO row inserted with status Pending	Pass
TC-13	Receive purchase order	Select pending PO, click Receive	Status set to Received and stock incremented atomically	Pass
TC-14	Receive already-received PO	Select received PO, click Receive	Error dialog, no database change	Pass

ID	Scenario	Input	Expected Output	Result
TC-15	Forecast with no sales history	Medicine with zero sales	predictDemand() returns 0, no false alert	Pass
TC-16	Forecast with known data	50 units sold over 30 days	Seven-day forecast = 12	Pass
TC-17	Expiry alert on dashboard	Medicine expiring in 20 days	Alert displayed with EXPIRY: prefix	Pass
TC-18	Inventory value on dashboard	Two medicines: 10 units x \$5 and 20 units x \$3	Total shown as \$110.00	Pass
TC-19	Launch without network Derby server	Server process not running	Application starts normally on embedded Derby	Pass
TC-20	First launch on clean machine	No existing PharmaDb database	All five tables created and default admin seeded	Pass

All 20 test cases passed. The SQL injection vulnerability identified in TC-02 is documented in Chapter 8 with a concrete fix. No other blocking defects were found during testing.

Security Analysis

8.1 Vulnerabilities and Fixes

SQL Injection in Login (Critical). The login query concatenates user-supplied input directly into a SQL string:

```
String selectQ = "select * from User1.AGENTS where A_NAME='"  
    + txtUserName.getText() + "' and A_PASSWORD = '" + pwd + "'";
```

An attacker entering `' OR '1'='1' --` as the username causes the query to return all rows regardless of the password. Since the code checks `Rs.next()` to determine login success, the attacker gains access as the first record in the AGENTS table, which is the Admin account. This is an OWASP Top 10 A03 injection vulnerability. The fix is straightforward: replace the concatenated string with a PreparedStatement:

```
PreparedStatement pstmt = conn.prepareStatement(  
    "SELECT * FROM User1.AGENTS WHERE A_NAME = ? AND A_PASSWORD = ?");  
pstmt.setString(1, txtUserName.getText());  
pstmt.setString(2, new String(l_password.getPassword()));
```

This change takes under an hour and completely closes the vulnerability. It should be applied before the system is used in any real pharmacy.

Plaintext Password Storage (High). Passwords are stored as plain text in the A_PASSWORD column. Anyone with read access to the Derby database files on disk can recover all credentials immediately. The fix is to hash passwords with BCrypt using the open-source jbcrypt library before storing them, and verify at login with `BCrypt.checkpw()`. A one-time migration routine should re-hash all existing plaintext passwords on the first upgraded launch.

No Session Timeout (Medium). An authenticated session remains open until the application is closed. An unattended workstation therefore gives full access to whoever sits at it with no re-authentication required. The fix is a `javax.swing.Timer` that fires after a configurable inactivity period and calls the logout action, returning the user to the login screen.

No Administrative Audit Log (Medium). The SALES table provides a complete log of billing transactions, but there is no record of administrative operations: medicine records deleted, passwords changed, supplier records modified. The fix is an AUDIT_LOG table with

columns for timestamp, agent name, action type, table name, and affected record ID, with a row inserted from every CRUD operation via a shared helper method.

Vulnerability	Severity	Fix Summary	Estimated Effort
SQL injection in login	Critical	Replace string-concatenated query with PreparedStatement	1 hour
Plaintext passwords	High	BCrypt hashing with jbcrypt, one-time migration of existing passwords	4 to 6 hours
No session timeout	Medium	Swing Timer returning to login screen after inactivity period	2 to 3 hours
No audit log	Medium	AUDIT_LOG table written from all CRUD operations	6 to 8 hours

Limitations and Future Work

Single-user embedded mode. The embedded Derby configuration supports only one active JVM process writing to the database at a time. A pharmacy with more than one computer must configure Derby as a network server, which adds a one-time setup step. Full concurrent multi-user access with proper connection pooling requires moving to a client-server architecture with a backend API layer.

Simple Moving Average forecasting accuracy. The SMA model treats all 30 days equally. It does not account for seasonal patterns (for example, antimalarial medicines sell at higher rates during Senegal's rainy season), day-of-week effects, promotions, or supply disruptions. The predictions are conservative and reliable for stable products but will underperform for medicines with variable demand cycles.

No foreign key constraints. Tables reference each other by name string rather than by numeric ID with a formal foreign key. Renaming a medicine or a supplier company does not cascade to SALES or PURCHASE_ORDERS records, which silently breaks the link without any database error being raised. This is the second item on the fix roadmap because it requires a schema migration to add ID-based FK columns to all affected tables.

No prescription management. Prescription-only medicines legally require a valid prescription before dispensing. The system has no mechanism to capture, store, or verify prescription information, which limits its use in any regulated or clinical setting.

Manual data entry only. Every stock entry and every sale requires the pharmacist to type all values. Barcode or QR scanning at the point of sale and at goods receipt would reduce data entry errors and save significant time during high-volume periods.

Priority	Feature	Technical Approach
High	Fix SQL injection and hash passwords	PreparedStatement everywhere, BCrypt with jbcrypt
High	Add proper foreign key constraints	Schema v2 migration: add ID columns and FK declarations
Medium	REST API backend	Spring Boot replacing direct Derby calls, enabling multi-client access
Medium	Web or mobile frontend	React frontend consuming the REST API
Medium	Barcode and QR scanning	ZXing library integrated with a USB or camera-based scanner
Low	Improved forecasting model	Holt-Winters exponential smoothing or LSTM neural network
Low	Multi-branch support	Branch table with branch FK on all entities and per-branch dashboard views
Low	Prescription management	PRESCRIPTIONS table linked to sales, PDF generation for dispensing records

Conclusion

The Pharmacy Tracking System delivers on all seven objectives stated at the outset. It manages medicine inventory with full batch information, records every sale and feeds that data into a demand forecasting engine, monitors expiry dates, handles purchase orders with proper transaction guarantees, and enforces role-based access control. It does all of this without requiring a server, a database administrator, or an internet connection, and it installs itself on first launch.

Three technical decisions proved most significant during development. First, centralising all JDBC calls in DatabaseHelper eliminated a large amount of duplicated connection-management code that was scattered across earlier versions of the frames. A change to a query now affects one location rather than five. Second, the transaction-based stock-receiving implementation in PurchaseOrderFrame is the correct engineering approach for any operation that must update more than one table atomically. The explicit setAutoCommit/commit/rollback pattern should be applied to any future multi-table operation added to the system. Third, the hybrid embedded-and-network Derby connection strategy, implemented as a simple try-catch in getConnection(), gives the system genuine flexibility: it works as a single-user desktop tool out of the box, and it scales to a shared pharmacy counter simply by starting the Derby network server, with no code changes required.

The security findings are the most urgent items on the development roadmap. The SQL injection vulnerability in the login screen is a five-line fix. The password hashing upgrade is a half-day of work. Both should be applied before this system is deployed in a real pharmacy. The remaining findings (session timeout, audit log) are medium-priority improvements that would be required for any regulated deployment.

With those fixes applied, this system is a practical and complete tool for the community pharmacy context it was designed for: affordable, offline-capable, easy to set up, and covering all the operational tasks that manual records and spreadsheets handle poorly.

Source code: <https://github.com/utachicodes/PharmacyTrackingSystem>

References

1. Apache Software Foundation. (2023). *Apache Derby Developer Guide, Release 10.16*. Apache Software Foundation.
2. Booch, G., Rumbaugh, J., and Jacobson, I. (2005). *The Unified Modeling Language User Guide*, 2nd edition. Addison-Wesley.
3. Diallo, M. and Sow, F. (2022). Digital readiness in Senegalese community pharmacies: a cross-sectional survey. *Journal of African Health Informatics*, 4(1), 12-24.
4. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
5. Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
6. Horstmann, C. and Cornell, G. (2019). *Core Java Volume I: Fundamentals*, 11th edition. Prentice Hall.
7. Narayana, M. R., Pillai, R. S., and Kaur, G. (2019). Comparative analysis of demand forecasting methods for pharmaceutical supply chains. *International Journal of Supply Chain Management*, 8(3), 201-211.
8. OWASP Foundation. (2021). *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. Open Web Application Security Project.
9. Okunade, K. S., Lawal, A. B., and Adeyemi, T. O. (2020). Pharmacy management practices in Lagos State, Nigeria. *West African Journal of Pharmacy*, 31(2), 44-53.
10. Oracle Corporation. (2023). *Java SE 17 API Documentation*. Oracle Technology Network.
11. Silver, E. A., Pyke, D. F., and Thomas, D. J. (2016). *Inventory and Production Management in Supply Chains*, 4th edition. CRC Press.
12. Toedter, K. (2006). *JCalendar: Java Date Picker Library*. <https://toedter.com/jcalendar/>
13. World Health Organisation. (2020). *Medicines Supply Management: A Reference Manual for Pharmaceutical Supply Managers*. WHO Press.

Key Source Code Excerpts

DatabaseHelper.java

```
public class DatabaseHelper {
    private static final String DB_URL = "jdbc:derby://localhost:1527/PharmaDb";
    private static final String USER = "User1";
    private static final String PASS = "User1";

    public static Connection getConnection() throws SQLException {
        try {
            return DriverManager.getConnection(DB_URL, USER, PASS);
        } catch (SQLException e) {
            return DriverManager.getConnection(
                "jdbc:derby:PharmaDb;create=true", USER, PASS);
        }
    }

    public static DefaultTableModel resultSetToTableModel(ResultSet rs) {
        try {
            ResultSetMetaData meta = rs.getMetaData();
            int cols = meta.getColumnCount();
            Vector<String> colNames = new Vector<>();
            for (int i = 1; i <= cols; i++)
                colNames.add(meta.getColumnLabel(i));
            Vector<Vector<Object>> rows = new Vector<>();
            while (rs.next()) {
                Vector<Object> row = new Vector<>();
                for (int i = 1; i <= cols; i++)
                    row.add(rs.getObject(i));
                rows.add(row);
            }
            return new DefaultTableModel(rows, colNames);
        } catch (Exception e) { e.printStackTrace(); return null; }
    }

    public static void initializeDatabase() {
        try (Connection conn = getConnection();
            Statement stmt = conn.createStatement()) {
            // Create MEDICINE - if it already exists, the catch block is a no-op
            try {
                stmt.execute("CREATE TABLE MEDICINE ( "
                    + "M_ID INT PRIMARY KEY, M_NAME VARCHAR(50), "
                    + "M_QUANTITY INT, M_PRICE DOUBLE, "
                    + "M_EXPDATE DATE, M_MFTDATE DATE, "
                    + "M_COMPANY VARCHAR(50), "
                    + "M_OWNER VARCHAR(50) DEFAULT 'Main', "
                    + "M_CATEGORY VARCHAR(50), M_STRENGTH VARCHAR(20), "
                    + "M_DOSAGE VARCHAR(50), M_UNIT_COST DOUBLE, "
                    + "M_THRESHOLD INT DEFAULT 10, M_BATCH VARCHAR(50))");
            } catch (SQLException e) { /* table exists, continue */ }
            // AGENTS, SALES, PURCHASE_ORDERS, COMPANY created the same way ...
            // Seed default admin on first launch
        }
    }
}
```

```
        try (ResultSet rs = stmt.executeQuery(
            "SELECT COUNT(*) FROM AGENTS")) {
            if (rs.next() && rs.getInt(1) == 0) {
                stmt.execute("INSERT INTO AGENTS VALUES "
                    + "(1,'Admin',30,'admin123','0000000000',"
                    + "'Other','admin@pharma.com','Admin')");
            }
        }
    } catch (SQLException e) { e.printStackTrace(); }
}
```

ForecastingHelper.java

```
public class ForecastingHelper {

    public static int predictDemand(String medicineName) {
        int totalQty = 0, count = 0;
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(
                "SELECT S_QTY FROM User1.SALES "
                + "WHERE S_MED_NAME = ? AND S_DATE >= ?")) {
            LocalDate ago = LocalDate.now().minusDays(30);
            pstmt.setString(1, medicineName);
            pstmt.setDate(2, java.sql.Date.valueOf(ago));
            try (ResultSet rs = pstmt.executeQuery()) {
                while (rs.next()) {
                    totalQty += rs.getInt("S_QTY");
                    count++;
                }
            }
            if (count == 0) return 0;
            // Divide by 30 (window), not count, so zero-sale days are averaged in
            return (int) Math.ceil((double) totalQty / 30.0 * 7);
        } catch (SQLException e) { e.printStackTrace(); return 0; }
    }

    public static List<String> getLowStockAlerts() {
        List<String> alerts = new ArrayList<>();
        try (Connection conn = DatabaseHelper.getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(
                "SELECT M_NAME, M_QUANTITY FROM User1.MEDICINE")) {
            while (rs.next()) {
                String name = rs.getString("M_NAME");
                int qty = rs.getInt("M_QUANTITY");
                int pred = predictDemand(name);
                if (qty < pred || qty < 10)
                    alerts.add(name + " (stock: " + qty
                        + ", 7-day forecast: " + pred + ")");
            }
        } catch (SQLException e) { e.printStackTrace(); }
        return alerts;
    }

    public static List<String> getExpirationAlerts() {
        List<String> alerts = new ArrayList<>();
        try (Connection conn = DatabaseHelper.getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(
                "SELECT M_NAME, M_EXPDATE FROM User1.MEDICINE")) {
            LocalDate limit = LocalDate.now().plusDays(30);
            while (rs.next()) {
                java.sql.Date d = rs.getDate("M_EXPDATE");
                if (d != null && d.toLocalDate().isBefore(limit))
                    alerts.add("EXPIRY: " + rs.getString("M_NAME")
                        + " (expires: " + d.toLocalDate() + ")");
            }
        }
    }
}
```

```
        } catch (SQLException e) { e.printStackTrace(); }
        return alerts;
    }

    public static double getInventoryValue() {
        double total = 0;
        try (Connection conn = DatabaseHelper.getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(
                "SELECT M_QUANTITY, M_UNIT_COST FROM User1.MEDICINE")) {
            while (rs.next())
                total += rs.getInt("M_QUANTITY") * rs.getDouble("M_UNIT_COST");
        } catch (SQLException e) { e.printStackTrace(); }
        return total;
    }
}
```